LESSON 15

Motor Mania

Spin a fan at any speed in both directions, then make a stepper motor move one tiny step at a time.

Name: _____ Date: _____

TODAY YOU WILL LEARN

- Why motors need their **own power supply**
- How a **motor driver chip** (L293D) lets the ESP32 control a motor
- How to change **speed** with PWM and **direction** with two pins
- How a **stepper motor** moves in exact steps

SESSION PLAN

0:00	Why can't the ESP32 power a motor?
0:10	Set up the power supply module
0:20	Wire the L293D and the fan motor
0:40	Sketch 1: **L15_Fan**
0:55	Short break
1:00	Wire the stepper, Sketch 2: **L15_Stepper**
1:25	Stepper experiments
1:35	Challenges
1:55	Quiz and tidy up

PARTS TO COLLECT

- ☐ ESP32 board + USB-C cable
- ☐ 2 breadboards
- ☐ Power supply module + 9V battery with clip (or a 2nd USB cable)
- ☐ L293D chip (16 legs)
- ☐ DC motor (3–6 V) + fan blade
- ☐ Stepper motor (28BYJ-48) + ULN2003 driver board
- ☐ Jumper wires (6 female-to-male for the stepper board)
- ☐ Extra: potentiometer (for a challenge)

1. Motors are hungry!

An ESP32 pin can give out a tiny amount of electricity, enough for an LED. A motor needs **10 to 20 times more**! If you connected a motor straight to a pin, it would hardly turn, and the ESP32 could be damaged.

So we split the job in two:

- The **ESP32** is the **brain**. It sends small "go/stop/faster" signals.
- A **driver chip** is the **muscle**. It takes those signals and switches a big current from a separate **power supply** through the motor.

Safety rules for today

- Motor power comes **only** from the power supply module, **never** from the ESP32's 3V3 or 5V pins.
- Switch the power module **off** while you change wires.
- Keep fingers and hair away from the spinning fan. Hold or tape the motor down.
- If anything gets hot or smells funny: switch off and unplug **everything**, then call your teacher.

2. Set up the power supply module

The power supply module plugs straight into the end of a breadboard. It turns a 9 V battery (or a USB cable) into a steady **5 V** on the breadboard rails.

✓	Step
☐	**Remove** any wires from the ESP32's **5V** or **3V3** pins to the breadboard rails. (Two power supplies must never be joined!)
☐	Push the module onto the end of one breadboard, so its pins sit in the **+** and **–** rails (match **+** to **+**!)
☐	Set **both** yellow jumpers on the module to **5V**
☐	Connect a wire from an ESP32 **GND** pin to the module's **–** rail
☐	Clip the 9 V battery to its plug and plug it into the module (or use a USB cable). Leave the switch **off** for now

> **Big idea: share the ground**
>
> The ESP32 and the power module have separate **+**, but they must share **GND**. GND is the "zero" that both measure their voltages from. Without a shared GND, the driver chip can't understand the ESP32's signals.

3. Wire the fan motor

The **L293D** chip has 16 legs. Find the little **notch** (a half-circle) at one end. With the notch at the top, leg 1 is top-left. Count **down** the left side 1 to 8, then **up** the right side 9 to 16.

Push the chip into a free part of a breadboard, **across the middle gap**, so each leg has its own row.

✓	From	To
☐	L293D leg **1** (Enable: speed)	ESP32 pin **25**
☐	L293D leg **2** (Input 1)	ESP32 pin **26**
☐	L293D leg **3** (Output 1)	one **motor** wire
☐	L293D legs **4** and **5** (GND)	**–** rail
☐	L293D leg **6** (Output 2)	the other **motor** wire
☐	L293D leg **7** (Input 2)	ESP32 pin **27**
☐	L293D leg **8** (motor power)	**+** rail (5 V from module)
☐	L293D leg **16** (chip power)	**+** rail (5 V from module)

Push the fan blade onto the motor's shaft. Check every wire twice, then switch the module **on**.

4. Sketch 1: Speed and direction

File → **New Sketch**, type this in, save as **L15_Fan**, upload, and open the **Serial Monitor**.

Type this in → save it as **L15_Fan** 41 lines

```
// Lesson 15: DC motor fan with the L293D chip
// EN (speed) on 25, IN1 on 26, IN2 on 27.

const int EN_PIN = 25;
const int IN1_PIN = 26;
const int IN2_PIN = 27;

void motor(int speed, bool forward) {
  digitalWrite(IN1_PIN, forward);
  digitalWrite(IN2_PIN, !forward);
  analogWrite(EN_PIN, speed);       // 0 = stop, 255 = full speed
}

void setup() {
  Serial.begin(115200);
  pinMode(EN_PIN, OUTPUT);
  pinMode(IN1_PIN, OUTPUT);
  pinMode(IN2_PIN, OUTPUT);
}

void loop() {
  Serial.println("Forward, slow");
  motor(150, true);
  delay(3000);

  Serial.println("Forward, fast");
  motor(255, true);
  delay(3000);

  Serial.println("Stop");
  motor(0, true);
  delay(2000);

  Serial.println("Backward");
  motor(200, false);
  delay(3000);

  Serial.println("Stop");
  motor(0, true);
  delay(2000);
}
```

How it works

Code	What it means
`IN1 HIGH`, `IN2 LOW`	Current flows through the motor one way: it spins **forwards**
`IN1 LOW`, `IN2 HIGH`	Current flows the other way: it spins **backwards**
`digitalWrite(IN2_PIN, !forward);`	IN2 is always the *opposite* of IN1
`analogWrite(EN_PIN, speed);`	**PWM** from Lesson 5: switch the motor on and off very fast. More "on" time = faster
`void motor(int speed, bool forward)`	Our own function with **two inputs**, so `motor(150, true)` does all the work

Try it

1. What is the **lowest** speed that still makes your fan start from standing still?

   ---

2. Does the fan blow air at you when it goes forwards, or backwards? How could you swap that **without** changing the code?

Motor doesn't spin?

Is the module switched on, with jumpers on 5V? Is the battery fresh? Did you connect ESP32 GND to the **–** rail? Try `motor(255, true)`. Some motors won't start at slow speeds.

5. The stepper motor

A **stepper motor** doesn't spin freely. It moves in tiny, exact **steps**. Inside are 4 electromagnets in a circle. Switch them on in the right order and the middle is pulled round, step by step. Our stepper needs **2048 steps** for one full turn. That's how printers and 3D printers move so precisely!

The **ULN2003** board is the stepper's driver (its "muscle"). Its 4 little LEDs show you which magnets are on.

✓	From	To
☐	Switch the power module **off** first!	
☐	Stepper's **white plug**	ULN2003 white socket (it only fits one way)
☐	ULN2003 **IN1**	ESP32 pin **16**
☐	ULN2003 **IN2**	ESP32 pin **17**
☐	ULN2003 **IN3**	ESP32 pin **18**
☐	ULN2003 **IN4**	ESP32 pin **19**
☐	ULN2003 **–**	**–** rail
☐	ULN2003 **+**	**+** rail (5 V from module)

Stick a small piece of tape on the shaft like a flag so you can see it turn. Switch the module **on**.

6. Sketch 2: Step by step

File → New Sketch, type this in, and save as **L15_Stepper**.

Type this in → save it as **L15_Stepper** — 50 lines

```
// Lesson 15: Stepper motor
// ULN2003 board IN1–IN4 on pins 16, 17, 18, 19.

const int PINS[4] = {16, 17, 18, 19};
const int PATTERN[4][4] = {
  {1, 1, 0, 0},
  {0, 1, 1, 0},
  {0, 0, 1, 1},
  {1, 0, 0, 1}
};
const int STEPS_PER_TURN = 2048;

int stepNumber = 0;

void step(int direction) {
  stepNumber = (stepNumber + direction + 4) % 4;
  for (int i = 0; i < 4; i++) {
    digitalWrite(PINS[i], PATTERN[stepNumber][i]);
  }
  delay(3);
}

void allOff() {
  for (int i = 0; i < 4; i++) {
    digitalWrite(PINS[i], LOW);
  }
}

void setup() {
  Serial.begin(115200);
  for (int i = 0; i < 4; i++) {
    pinMode(PINS[i], OUTPUT);
  }
}

void loop() {
  Serial.println("One full turn forwards");
  for (int i = 0; i < STEPS_PER_TURN; i++) {
    step(1);
  }
  allOff();
  delay(1000);
```

```
  Serial.println("Quarter turn backwards");
  for (int i = 0; i < STEPS_PER_TURN / 4; i++) {
    step(-1);
  }
  allOff();
  delay(1000);
}
```

Code	What it means
`const int PATTERN[4][4]`	A **2D array**: a table with 4 rows and 4 columns. Each row says which of the 4 magnets is on
`PATTERN[stepNumber][i]`	Row `stepNumber`, column `i`
`(stepNumber + direction + 4) % 4`	`%` gives the **remainder** after dividing. It makes 0,1,2,3 go round in a circle: after 3 comes 0 again
`step(-1)`	Go through the pattern **backwards**, so the motor turns the other way
`delay(3);`	Wait 3 ms between steps. Too short and the motor can't keep up
`allOff()`	Switch all magnets off so the motor doesn't get warm while it waits

Experiments

1. Watch the 4 LEDs on the ULN2003 board. What pattern do you see? (Try `delay(300)` in `step()` to slow it right down.)

   ---

2. What is the smallest delay that still works? What happens if you go lower?

   ---

3. How many steps make **half** a turn? How many steps for 90°?

   ---

7. Challenges

Challenge 1: Fan control knob

Add the potentiometer (middle leg to pin **34**, outer legs to ESP32 **3V3** and **GND**). Knob in the middle = stop, turn right = forwards, turn left = backwards. Turn further = faster.

Challenge 2: Turn to an angle

Write a function `turnDegrees(int degrees)` that turns the stepper by any angle. Positive = forwards, negative = backwards. *Hint: steps = degrees × 2048 ÷ 360.*

Challenge 3: Stepper clock

Make a paper clock face and a paper seconds hand. Make the stepper go round **exactly once per minute**, ticking every second.

8. Quiz

Check what you learned

1. Why don't we connect a motor straight to an ESP32 pin?

2. Which two things do we change to make the fan spin the other way?

3. What is `10 % 4`?

Tidy up: switch the power module **off** and unplug the battery **first**, then unplug the USB cable. Unclip the battery so it doesn't run flat.